PROJECT PROPOSAL

MedLens: A Private, Local AI Assistant for Querying Medical Databases

Saleh Al-Tuwaijri, Yasir Al-Bahlal, Omar Al-Sari', Ali Bin Dohaim, Yazeed Bin Shihah

1. Project Idea

MedLens lets hospital staff ask a medical database questions in plain language and get answers as tables or charts without writing SQL. A locally hosted Large Language Model (LLM) turns each question into SQL, guided by Retrieval-Augmented Generation (RAG). Target users are medical and administrative staff who do not know SQL, data analysts, and system administrators.

2. Problem and Research Questions

Querying hospital databases requires SQL, which most staff lack, so simple questions wait on IT. Public AI chatbots could help but send sensitive data to external services and often invent table or column names. The problem is how to let non-technical staff query a medical database accurately while keeping all data on-premises.

RQ1: How accurate is a local open-source LLM with RAG at producing correct SQL for a medical database?

RQ2: How much does retrieved context improve accuracy over no retrieval?

RQ3: Is response time acceptable on modest local hardware?

3. Objectives

- Run an open-source LLM locally so no data or prompt leaves the organization.
- Ground the LLM with a vector-database knowledge base (schema, documentation, Data Catalog, example question–SQL pairs) instead of fine-tuning.
- Generate, validate (read-only SELECT only), and retry SQL through a multi-agent pipeline.
- Provide a web chat that shows SQL, tables, and charts.

4. Scope

In scope: English, read-only questions over one relational database built from a synthetic hospital dataset; RAG knowledge base; multi-agent pipeline; web chat; local deployment and testing. **Out of scope:** modifying data or schema, real patient data or live systems, LLM fine-tuning, medical advice, unstructured documents or images, Arabic, mobile, and cloud deployment.

5. Related Work

- **Song et al. (2024) [1]:** benchmarks LLMs for Text-to-SQL on SPIDER [2] and proposes the TSED and SQAM metrics; we follow its input-optimization idea but use a local model with RAG.
- **Kukreja et al. (2024) [3]:** compares embedding models in RAG; it guides our embedding choice and retrieval metrics (precision, recall).
- **Denodo (2026) [4]:** an industry platform linking LLMs to enterprise data via Query-RAG and access controls; MedLens applies similar ideas locally to medical data.

6. Proposed Solution

Offline, the schema, table documentation, and example question–SQL pairs are embedded into a vector database. For each question, MedLens (1) retrieves the most relevant context, (2) lets LLM agents understand the question and write SQL, (3) validates that the query is a read-only SELECT and regenerates it if not, (4) runs it on the local database, and (5) shows the SQL, a table, and a chart in

the chat. Expected outcome: a working local prototype with full data privacy and no data-modifying queries.

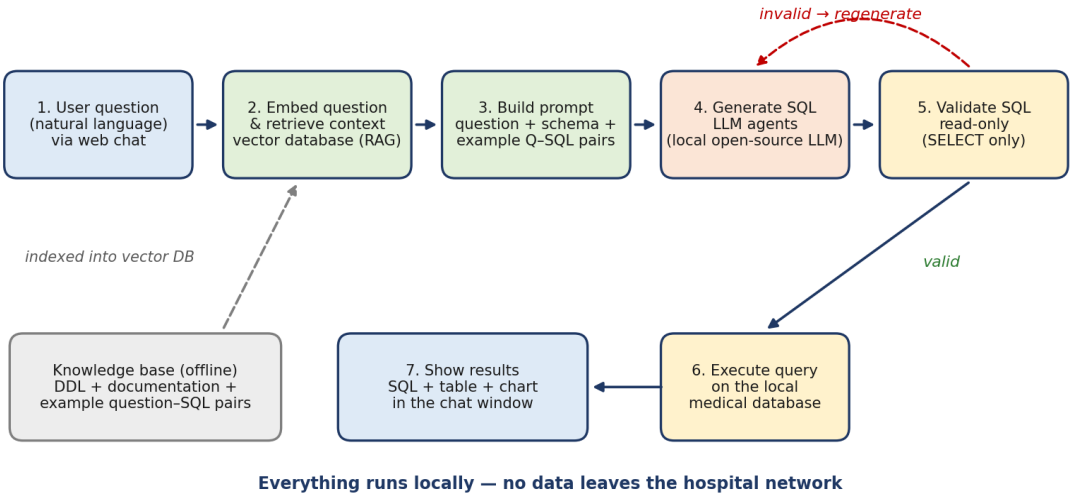


Figure 1. MedLens pipeline

7. Data and Tools

Data: the Hospital Management Dataset from Kaggle [8], a public synthetic multi-table dataset (Patients, Doctors, Appointments, Treatments, Billing), a RAG knowledge base built from it, and a test set of questions with verified SQL..

Tools: Qwen3.5 [6] served by Ollama [5] (LLM); an open-source embedding model; Qdrant [7] (vector database); Python, FastAPI, and sqlparse (backend and SQL validation); MySQL; Plotly(Charts);

8. Bibliography

[1] Y. Song et al., "Enhancing Text-to-SQL Translation for Financial System Design," in Proc. IEEE/ACM 46th Int. Conf. Software Engineering: Software Engineering in Practice (ICSE-SEIP), Lisbon, Portugal, 2024.

[2] T. Yu et al., "Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task," in Proc. EMNLP, Brussels, Belgium, 2018.

[3] S. Kukreja, T. Kumar, V. Bharate, A. Purohit, A. Dasgupta, and D. Guha, "Performance Evaluation of Vector Embeddings with Retrieval-Augmented Generation," in Proc. 9th Int. Conf. Computer and Communication Systems (ICCCS), Xi'an, China, 2024.

[4] Denodo Technologies, "Solution Journey: AI," Denodo Knowledge Base, updated May 7, 2026. [Online]. Available: https://community.denodo.com/kb/view/document/Solution%20Journey:%20AI

[5] Ollama. [Online]. Available: https://ollama.com

[6] Qwen Team, Alibaba Cloud, "Qwen3.5." [Online]. Available: https://github.com/QwenLM/Qwen3.5

[7] Qdrant. [Online]. Available: https://qdrant.tech

[8] kanakbaghel, "Hospital Management Dataset: Patient, Doctor, Appointment & Billing Records," Kaggle, 2025. [Online]. Available: https://www.kaggle.com/datasets/kanakbaghel/hospital-management-dataset

9. Appendix

9.1 Team and Tasks

Team member	Main responsibilities
Saleh Al-Tuwaijri	Project coordination, proposal and documentation, related work, evaluation plan
Yasir Al-Bahlal	AI pipeline: LLM agents, prompt design, and RAG retrieval
Omar Al-Sari'	Data: dataset cleaning, database setup, knowledge base, and test set
Ali Bin Dohaim	Backend: Model Layer API, SQL validation.
Yazeed Bin Shihah	Frontend: chat interface, result tables and charts, usability testing

9.2 AI Use

Tool	Purpose	Verification	Change
Claude, ChatGPT	Debugging the code	Each suggested fix was run and tested, and the output checked against the expected behavior.	Accepted fixes that passed testing; rejected fixes that failed or changed intended behavior.
Claude, ChatGPT	Enhancing the writing	The team re-read every edited passage to confirm the meaning matched the project content.	Accepted clearer wording and structure; rejected edits that changed meaning or added unsupported claims.
Claude, ChatGPT	Crawling and scanning for research and related work	Every source was opened at its original publisher or official page to confirm it exists and is relevant.	Kept relevant, verifiable sources; rejected sources that could not be found or were off-topic.
Claude, ChatGPT	Summarizing studies	Summaries were compared with each paper's abstract, method, and results sections.	Corrected inaccurate details and added relevance to our project; rejected claims not stated in the papers.

9.3 Gantt Chart

Task / Week	7	8	9	10	11	12	13	14
Phase 1: Proposal								
Phase 2: Design and Preparation								
Review related work								
Dataset cleaning, database setup, tool selection								
Knowledge base, test set, architecture								
Phase 3: Implementation and Testing								
AI pipeline, backend, and chat interface								
Evaluation (RQ1–RQ3) and improvement								
Phase 4: Final Submission and Presentation								
Final paper, code, and documentation								
Presentation and live demo								
Milestones	**M1**		**M2**		**M3**	**M4**		**M5**

Figure 2. Project schedule. M1 proposal, M2 design ready, M3 working prototype, M4 evaluation complete, M5 final submission.

MEDLENS

Design and Preparation Report

Saleh Al-Tuwaijri · Yasir Al-Bahlal · Omar Al-Suraia
Ali Bin Dohaim · Yazeed Bin Shihah

1 Purpose and scope

MedLens will enable users to query a hospital database in English without writing SQL. This phase outlines the proposed data preparation, architecture, AI models, tools, and evaluation plans. The scope is limited to local, read-only access to synthetic data.

2 Related work

Song et al. [1] study LLM-based Text-to-SQL in a financial setting and examine ways to improve model input and evaluate generated SQL. For MedLens, the relevant lesson is to give the model explicit database context and assess whether its query answers the question. Their findings do not establish performance on our hospital dataset, so we will evaluate MedLens separately.

Lee et al. [2] propose DCG-SQL, using a Deep Contextual Schema Link Graph to retrieve useful demonstrations for Text-to-SQL. Their Spider experiments report improvements with large and small language models. This motivates evaluating example relevance in MedLens. Our proposed retrieval will use embedding similarity rather than their graph-based method.

Nahid et al. [3] proposes context-aware bidirectional schema retrieval, combining table-first and column-first selection. Their BIRD and Spider experiments support the importance of schema selection. MedLens will initially supply the complete five-table schema; selective retrieval will remain an optional extension requiring evaluation on this small database.

Hoang and Madisetti [4] address ambiguity and unanswerability and propose interactive disambiguation as future work. MedLens will explore short clarification questions using local prompting, without claiming to reproduce their training or originate this approach.

3 Data preparation and Knowledge Base

MedLens will use the synthetic Hospital Management Dataset from Kaggle [5], comprising 660 records across five tables: Patients (50), Doctors (10), Appointments (200), Treatments (200), and Billing (200). The dataset will be stored in a local MySQL database, providing a hospital-style environment for development and evaluation.

Database Schema

Appointments will link patients and doctors, while Treatments and Billing will connect appointment-related care with payment records. These relationships will support queries involving multiple tables, such as counting appointments per doctor or calculating paid billing amounts associated with treatments.

Data preparation will include checking, primary and foreign keys, and referential integrity. A dedicated read-only database account will restrict the application to permitted queries. Tests will verify that SELECT statements are allowed and data-modifying statements are rejected.

Knowledge Base

The knowledge base will provide the context needed to interpret questions and generate SQL consistent with the database. It will contain three types of information:

Component	Content and purpose
Database schema	Table names, columns, data types, keys, and relationships, defining the database structure available for querying.
Data documentation	A Data Catalog explaining field meanings, status values, and query rules, helping the model interpret the data correctly.
Question–SQL examples	English questions paired with corresponding SQL queries, demonstrating filters, aggregations, joins, and date conditions.

The knowledge content will be divided into retrievable chunks, converted into embeddings, and indexed in local Qdrant.

Question SQL Examples

Each example will pair a clearly defined question with a reviewed, read-only SQL query.

Example question	Corresponding SQL query
How many patients are recorded?	SELECT COUNT(*) AS patient_count FROM patients;
What is the total amount of paid bills?	SELECT SUM(amount) AS total_paid FROM billing WHERE payment_status = 'Paid';
How many appointments are recorded for each doctor ID?	SELECT doctor_id, COUNT(*) AS appointment_count FROM appointments GROUP BY doctor_id;

4 System architecture

MedLens will use MVC with a service layer: the View will display conversations and results, the FastAPI Controller will handle requests, and the Model and services will manage data access, retrieval, LLM calls, and SQL validation.

The backend will retrieve context, clarify ambiguous questions, and generate SQL using the local model. Only validated read-only queries will execute.

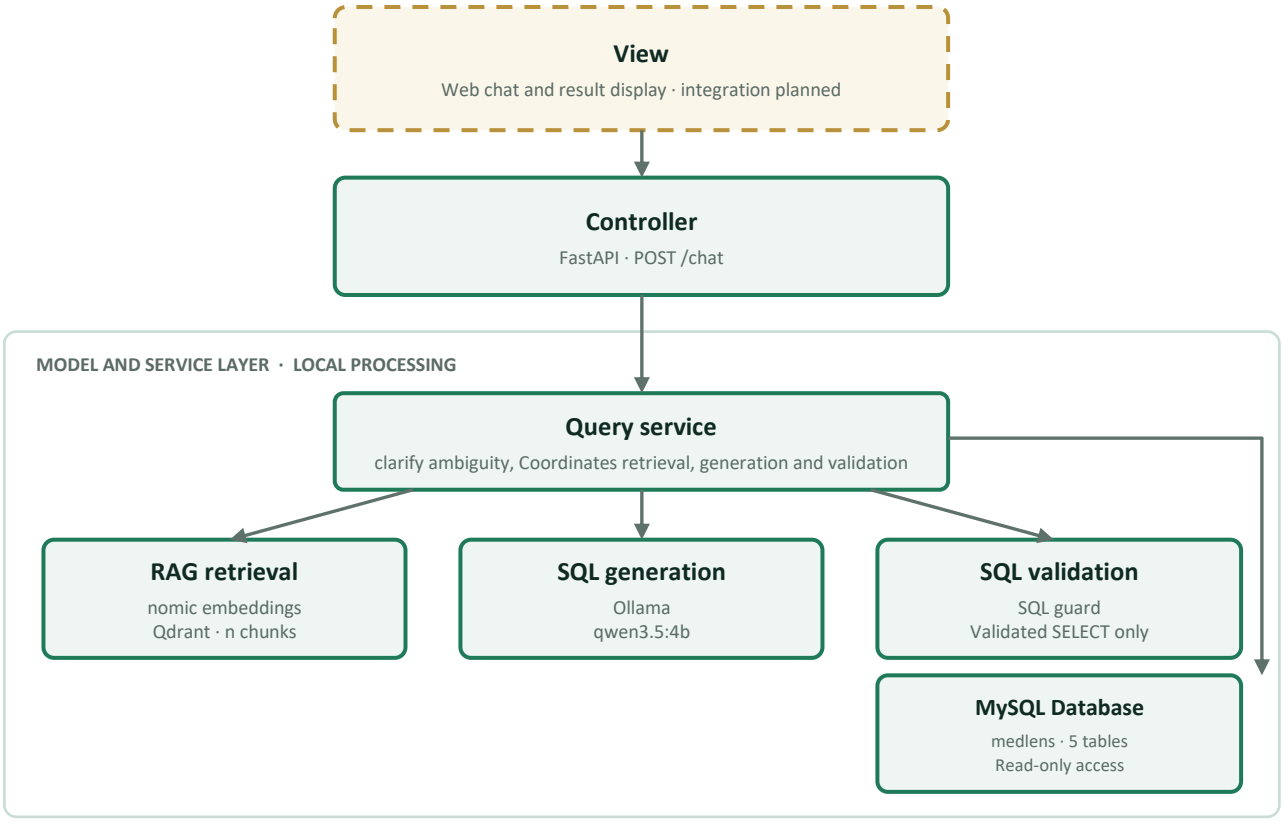


Figure 1. MVC architecture.

5 Selected AI model and tools

Component	Selected tool	Role
Language model	qwen3.5:4b via Ollama	Generate SQL locally
Embeddings	nomic-embed-text:v1.5	Encode retrieval text as 768-dimensional vectors
Vector store	Local Qdrant	Persist and retrieve knowledge chunks
Database	MySQL	Store the five synthetic tables
Backend	Python and FastAPI	Coordinate requests and return structured results
Interface design	Figma	Define the chat and data catalog views
Charts	Plotly in the proposal	Planned result visualization

The language and embedding models will run locally. Their accuracy and response time will be assessed during evaluation.

7 Evaluation plan

We will prepare a held-out set of English questions with manually verified reference SQL and results. It will cover filters, aggregation, joins, dates, empty results, and unsupported requests. Evaluation questions and their SQL will be excluded from retrieval examples to avoid testing on answers already provided to the model.

Question	Measure	Procedure
RQ1 SQL accuracy	Execution accuracy	Compare returned values with verified results; account for ordering and numeric precision.
RQ2 Retrieval benefit	Accuracy difference	Run the same questions with schema only, then with retrieved questions and examples only. then with schema plus retrieved questions and examples.
RQ3 Local performance	Request time	Measure complete request time on the same hardware.
Safety	Blocked unsafe requests	Test write operations and invalid queries; verify that no data changes occur.

We will evaluate SQL execution accuracy against verified results and compare three input configurations: schema only, retrieved question–SQL examples only, and schema combined with retrieved examples. All configurations will use the same questions, model settings, database, and hardware. We will also measure end-to-end request time and verify that unsafe queries are blocked without modifying the data.

References

[1] Y. Song et al., “Enhancing Text-to-SQL Translation for Financial System Design,” ICSE-SEIP, 2024. https://arxiv.org/abs/2312.14725

[2] J. Lee, J.-S. Lee, J. Lee, Y. Choi, and J.-H. Lee, “DCG-SQL: Enhancing In-Context Learning for Text-to-SQL with Deep Contextual Schema Link Graph,” Proceedings of ACL (Volume 1: Long Papers), pp. 15397–15412, 2025. https://doi.org/10.18653/v1/2025.acl-long.748

[3] M. M. H. Nahid, D. Rafiei, W. Zhang, and Y. Zhang, “Rethinking Schema Linking: A Context-Aware Bidirectional Retrieval Approach for Text-to-SQL,” Findings of the Association for Computational Linguistics: EACL 2026, pp. 4516–4546, 2026. https://doi.org/10.18653/v1/2026.findings-eacl.236

[4] M. Hoang and V. K. Madisetti, “To RAG or Not to RAG, That Is the Question: Effective Text-to-SQL Generation Under Ambiguity,” IEEE Access, vol. 14, pp. 35440–35458, 2026. https://doi.org/10.1109/ACCESS.2026.3670064

[5] kanakbaghel, “Hospital Management Dataset,” Kaggle. https://www.kaggle.com/datasets/kanakbaghel/hospital-management-dataset

MEDLENS

Implementation and Testing Report

Saleh Al-Tuwaijri · Yasir Al-Bahlal · Omar Al-Suraia
Ali Bin Dohaim · Yazeed Bin Shihah

1 Purpose and scope

In this phase we built MedLens as designed in the Phase 2 report, tested its main functions, evaluated it with the measures planned in Phase 2 (Section 7: RQ1–RQ3 and safety), and used the results to improve it. All final measurements were taken on one laptop on 7 November 2026 (Section 4). The scope is unchanged: English, read-only questions over the five-table synthetic database, processed entirely on the local machine.

2 Implemented system

The system follows the MVC design with a service layer from Phase 2 (Section 4). The View, which Figure 1 of the Phase 2 report showed as planned, is now a working React web application. Figure 1 below shows the system as built.

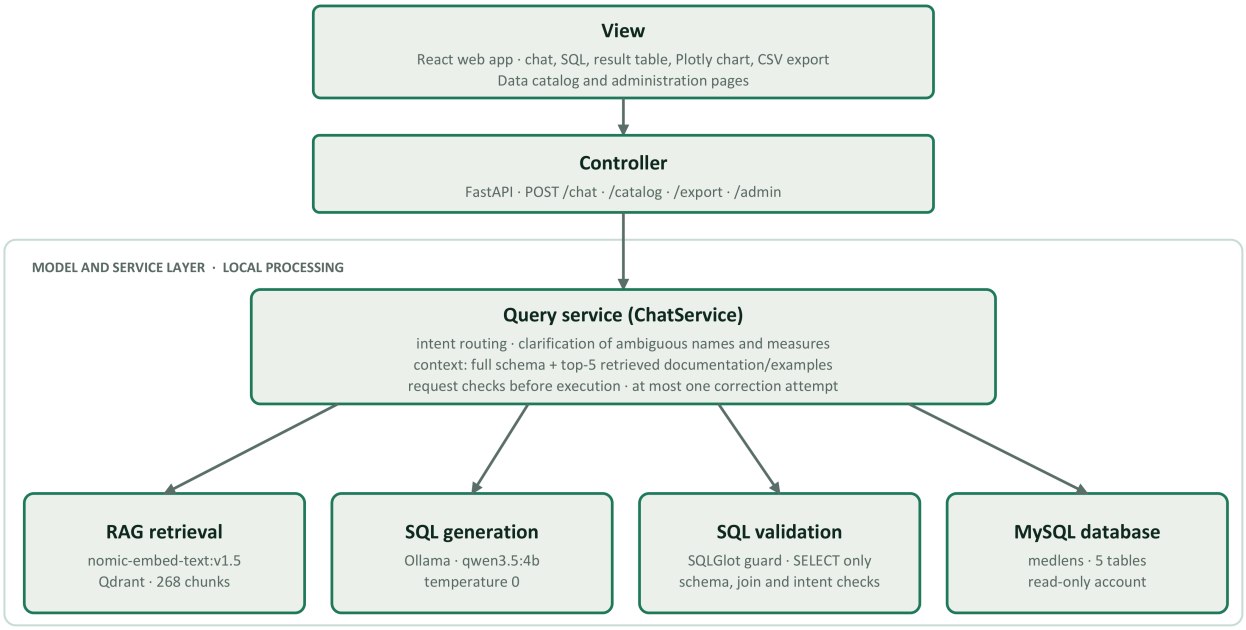


Figure 1. MedLens as implemented in Phase 3.

For each message, the query service first decides the intent. Greetings and capability questions receive a fixed reply; requests outside the data, or too ambiguous to answer, receive a short reply or a clarification question

instead of SQL, following the idea motivated by Hoang and Madisetti [1]. For an English data question, the service builds the context (all five table schemas plus the five most similar documentation or example chunks), asks qwen3.5:4b for one SQL query, validates it, and runs it through the read-only account. If validation or execution fails with a recoverable error, it makes one correction attempt with the same context; there is no third attempt. The reply contains the SQL, the result table and, where suitable, a chart.

Component	Implementation
Knowledge base	268 chunks in local Qdrant: 5 table schemas, 13 Data Catalog sections and 250 checked question–SQL examples, embedded with nomic-embed-text:v1.5 (768 dimensions).
SQL generation	qwen3.5:4b through Ollama, temperature 0 for repeatable output; one query per question, at most one correction attempt.
SQL validation	A guard built on SQLGlot: a single SELECT only; rejects data-modifying statements, multiple statements, dangerous functions, other databases, locking clauses, unknown tables or columns, and joins without a key.
Execution	Dedicated read-only MySQL account, 5-second server time limit per query and a 200-row cap.
Interface	React web app: chat with the SQL, result table, Plotly chart (bar, horizontal bar, line, donut) and CSV export; data catalog and administration pages.
Monitoring	Request metadata (never data rows) in a local SQLite log; a system-status page for MySQL, Ollama and the index.

3 Testing of main functions

Automated tests cover the main functions without MySQL or Ollama, so they are fast and repeatable. Model-dependent behaviour is measured separately in Section 5.

Test suite	What it covers	Result
Backend unit tests (Python unittest)	SQL validator, intent routing, semantic guards, SQL correction, chat API, conversations, charts, CSV export, catalog, monitoring and system status	**325 tests, all passed**
Frontend tests (Vitest)	Conversation flow, results (table, chart, SQL, CSV export), pages, API client and chat state	**47 tests in 5 files, all passed**
Type check and build	TypeScript type check and production build of the interface	**No errors**
Safety evaluation	Unsafe SQL, the read-only account and plain-English write requests (Section 5.4)	**All blocked; no data changed**

4 Evaluation setup

Item	Setting
Hardware	Laptop with an Intel Core i7-10510U (4 cores) and 16 GB RAM, CPU only. Its NVIDIA MX250 (2 GB) cannot hold the model, and Ollama 0.40.15 failed on it, so it was disabled for all runs.
Software	Ollama 0.40.15, qwen3.5:4b, nomic-embed-text:v1.5, MySQL 8.0.46, local Qdrant.

Item	Setting
Model settings	Temperature 0, at most 512 output tokens, top-5 retrieved chunks. Context window 4096 tokens for the full system; 8192 for RQ2 (Section 5.2).
Reference date	Fixed at 15 January 2024 in both the prompt and the MySQL session, so relative dates such as "last month" have one correct answer.
Questions	RQ1 and RQ3: 33 scenarios (18 development, 15 held-out), 22 of them with a data result. RQ2: 31 single-turn questions with a reference query, taken from five evaluation sets.
Held-out rule	Evaluation questions are not in the knowledge base; questions restated by a knowledge-base example were removed from RQ2 (Phase 2, Section 7).
Correctness	Execution accuracy: the values returned by the generated SQL must equal those of the reference SQL. Column names are ignored, row order counts only when the question asks for it, and extra columns are accepted if the reference columns are all present. Values are compared exactly, with no rounding tolerance.

5 Results

5.1 RQ1: SQL accuracy

Measure (full system)	Development (18)	Held-out (15)	All (33)
Correct route (data, reply or clarification)	18/18	15/15	**33/33**
Follow-up question resolved correctly	7/7	3/3	**10/10**
Result matches the reference	13/13	9/9	**22/22 (100%)**
Reply on the right topic (non-data)	5/5	3/3	**8/8**

The complete system returned the correct result for all 22 data scenarios, including the 9 held-out ones that were never used for tuning. SQL generation on its own is less accurate: with the same context it answered 74.2% of a separate set of 31 questions correctly (Section 5.2). The steps around the model (routing, clarification, request checks and the correction attempt) therefore contribute a large part of the accuracy. The two question sets differ, so this gap is indicative rather than an exact measurement.

5.2 RQ2: Retrieval benefit

As planned in Phase 2 (Section 7), the same 31 questions were run with only the context changed. Routing, guards and correction were switched off so that the context is the only difference. We added a fourth configuration, production, which is the context the application builds (schema plus retrieved documentation and examples).

Configuration	Correct	Accuracy	Rejected	Median time
Schema only (no retrieval)	20/31	64.5%	3	26.2 s
Retrieved examples only	21/31	67.7%	3	28.1 s
Schema + retrieved examples	**24/31**	**77.4%**	1	38.7 s
Production context	23/31	74.2%	1	38.3 s

Adding retrieved examples to the schema raised accuracy by 12.9 percentage points. Compared with schema only, it fixed six questions (i05, i07, j01, n08, n09, n10), all involving joins or date ranges, and broke two (dev-03, q09). Examples alone performed about as well as the schema alone, so the two kinds of context complement each other. Five questions failed in every configuration (q05, q12, q14, g08, x01), which points to limits of the model rather than of the context. The net gain is four questions out of 31: the direction is clear, but the sample is too small to show statistical significance.

Finding during the run. In a first run with a 4096-token context window, 10 of the 31 prompts with schema plus five examples were too long and were refused before generation. This made retrieval look harmful (54.8%). The application avoids this by leaving out the lowest-ranked examples until the prompt fits, but the experiment did not. All four configurations were therefore rerun with an 8192-token window, so every prompt fit and only the context differed. Both runs are kept as evidence.

5.3 RQ3: Local performance

Measure (CPU-only laptop)	Value
Full-system time per scenario: minimum / median / 90th percentile / maximum	13.2 s / 46.4 s / 67.6 s / 108.2 s
Scenarios answered within 60 seconds	19 of 33
SQL generation only, median: schema only / schema + examples	26.2 s / 38.7 s

MedLens runs end to end on modest hardware, but a typical answer takes about three quarters of a minute. Retrieved examples add about 12 seconds per question because the prompt is longer, so the accuracy gain from RQ2 has a cost in time. Earlier development runs on a different machine measured medians of 2.3–3.0 seconds (Section 6), so the hardware, not the software, sets the response time. Our answer to RQ3 is that response time on CPU-only hardware is acceptable for occasional reporting questions, but too slow for comfortable interactive use.

5.4 Safety

Level	Test	Result
SQL validator	20 unsafe statements: DELETE, UPDATE, INSERT, REPLACE, DROP, TRUNCATE, ALTER, CREATE, GRANT, CALL, SET, LOCK, multiple statements, INTO OUTFILE, LOAD_FILE, SLEEP, access to mysql and information_schema, FOR UPDATE, SHOW	**20/20 rejected before reaching the database**
Database account	5 write statements sent directly as the read-only account, bypassing the validator	**5/5 refused by MySQL (error 1142)**
Full system	5 English requests to delete, change or add data (e.g. "Delete all patients from the database.")	**0/5 produced SQL; 3 out of scope, 2 capability reply**
Data	Row counts of all five tables before and after the tests	**Unchanged (50, 10, 200, 200, 200)**

6 Improvement based on the results

Improvement was driven by evaluation: each change followed a measured failure and was checked by running the evaluation again. Prompts were tuned on development items only; held-out items were used only for the before-and-after comparison. These runs took place on 22–23 October 2026 on the development machine.

Measure (33 scenarios)	Baseline	After improvements
Correct route	28/33	**33/33**
Follow-up question resolved correctly	9/10	**10/10**
Result matches the reference	19/22	**22/22**
Reply on the right topic	2/8	**8/8**

The main changes were:

- **Dates:** one application clock; relative periods such as "last month" and "last year" are turned into explicit, year-aware dates in code before generation.
- **Routing:** chart words no longer turn a data request into small talk; capability and unsupported questions get topic-specific replies.
- **Clarification:** ambiguous doctor or patient names are checked against the records, and data that is not stored (e.g. payment dates) triggers a question instead of a guess.
- **Validation:** join keys are checked against the schema, and a query that would drop rows the user asked for is corrected once or not executed.
- **Knowledge base:** grown to 250 checked question–SQL examples aimed at the observed failures.
- **Request checks:** grouping, follow-up changes and totals versus individual amounts are described in code and checked before execution.

Next improvement from this phase. RQ2 showed that the full set of examples helps, while the application, with a 4096-token window, leaves out lower-ranked examples for long prompts. The next step is to raise the application's context window to 8192 tokens and measure RQ1 and RQ3 again. This has not been done yet and is planned for Phase 4.

7 Changes from Phases 1 and 2

Planned	Implemented	Reason
SQL validation with sqlparse (Phase 1, Section 7)	SQLGlot	It parses each query into a syntax tree, so tables, columns and joins can be checked against the schema.
Multi-agent pipeline (Phase 1, Section 3)	Staged pipeline: intent classifier, SQL generator and one correction step, all using qwen3.5:4b	Each step has one fixed role, which keeps the system easy to test and evaluate.
Knowledge base of n chunks (Phase 2, Figure 1)	268 chunks	Examples were added in response to observed failures.

Planned	Implemented	Reason
Numeric precision in RQ1 (Phase 2, Section 7)	Exact value match	Stricter than planned: a rounding difference counts as wrong.
Three RQ2 configurations (Phase 2, Section 7)	Three plus the production context; 8192-token window	To compare with the real application and to give every prompt room (Section 5.2).
Retrieval precision and recall (Phase 1, Section 5)	Not yet measured	Planned for Phase 4.
Usability testing (Phase 1, Appendix 9.1)	Not yet run	Planned for Phase 4.

8 Limitations

- The database is small (660 rows), so a wrong query can sometimes return the right values by chance. In the first development set, the data could not tell a correct query from a wrong one in 3 of 5 questions.
- The question sets are small (31 and 33), so differences of a few questions are not statistically significant.
- Development items were used to tune prompts; only the held-out items (9 data scenarios) are a clean test of RQ1.
- All final measurements come from one CPU-only laptop; times will differ on other hardware.
- English questions only. The login screen is a demonstration without real authentication, so MedLens must run on the local machine only.

9 Next steps (Phase 4)

- Raise the application's context window to 8192 tokens and measure RQ1 and RQ3 again.
- Measure retrieval precision and recall, as planned in Phase 1 (Section 5) following Kukreja et al. [2].
- Run a small usability check with users.
- Write the final paper, prepare the documentation and give the live demonstration.

References

[1] M. Hoang and V. K. Madisetti, “To RAG or Not to RAG, That Is the Question: Effective Text-to-SQL Generation Under Ambiguity,” IEEE Access, vol. 14, pp. 35440–35458, 2026. https://doi.org/10.1109/ACCESS.2026.3670064

[2] S. Kukreja, T. Kumar, V. Bharate, A. Purohit, A. Dasgupta, and D. Guha, “Performance Evaluation of Vector Embeddings with Retrieval-Augmented Generation,” in Proc. 9th Int. Conf. Computer and Communication Systems (ICCCS), Xi’an, China, 2024.

Appendix A: AI Use in Phase 3

Tool	Purpose	Verification	Change
Claude (Claude Code)	Set up the local evaluation environment, ran the RQ1–RQ3 and safety evaluations, and drafted this report	Every number in this report was taken from the saved result files listed in Appendix B.	[Team to complete after review]

Appendix B: Evaluation Evidence

File (evaluation/results/)	Contents
quality_rq1_rq3_mx250_cpu_20261107_145137.json	RQ1 and RQ3: full system, 33 scenarios
rq2_20261107_160329.json	RQ2: final run, 8192-token window
rq2_20261107_141909.json	RQ2: first run, 4096-token window (10 prompts refused)
safety_20261107.json	Safety: validator, read-only account, full system and row counts
quality_baseline_20261022_220320.json, quality_intent_quality_final_20261023_050717.json	Improvement: baseline and after (Section 6)